Programmation Logique

M. RAMDANI

ILISI 1

PLAN

- LOGIQUE DES PROPOSITIONS
- SYSTEMES FORMELS
- LOGIQUE DU PREMIER ORDRE
- PROLOG

Définition

Definition (Logique (*n. f.*))

- *Étude scientifique des conditions de vérité des propositions*
- *Manière de raisonner*
- *Enchaînement cohérent d'idées*

Introduction à la logique

Historique

De l'antiquite a la logique moderne

- Aristote
  - Notion de **prédicat** : « S est P »
  - Syllogisme : « $A \longrightarrow B$ et $B \longrightarrow C$ donne $A \longrightarrow C$ ».
- XIVe siècle
  - Buridan : généralisation de la logique d'Aristote
- XIXe siècle
  - Boole et Frege détachent la logique de la philosophie et la rattachent aux mathématiques

declic

passage de l'**implicite** à l'**explicite**

Introduction à la logique

Dans l'antiquité
Pour Aristote :
- Logique = instrument du savoir, non le savoir lui-même.
- Étudier les lois du discours
- Distinguer les raisonnements corrects de ceux incorrects

Années 50
- Volonté de mécaniser/automatiser la pensée rationnelle
- Développement de programmes de démonstration de théorèmes
- Quelques succès mais rapidement, essoufflement

Introduction à la logique

Années 70
Nouveaux résultats :
- Exécution d'un programme = effectuer une déduction logique
- Programme = preuve de l'obtention d'un théorème (le résultat) à partir d'autres théorèmes (les données initiales).
- La logique : outil conceptuel pour décrire l'action d'un ordinateur et définir des algorithmes.

1972 : Apparition de PROLOG, démonstrateur de théorèmes

Introduction à la logique

- Etudier les mécanismes déductifs permettant de tirer des conclusions à partir d'hypothèses (prémisses données),
c'est-à-dire d'enchainer toutes les étapes d'inférence.
- Essayer dans la mesure du possible de mécaniser la pensée.

Logique des propositions

Notion de proposition

Definition

Une **proposition** *est un énoncé du langage ordinaire considéré du point de vue formel. Cet énoncé est soit* **vrai** *soit* **faux** *mais pas les deux.*

Exemple

« Le chat du voisin est mort » est une proposition

Logique des propositions

Notion de valeur de vérité

- **Valeur de Vérité**

Proposition **vraie** si adéquation entre proposition et faits du monde réel, **fausse** si non

- **Exemples :**

Le lion est un carnivore

La somme de un et Deux est cinq

Logique des propositions

Étude du calcul propositionnel

Quatre étapes

- Comment écrire les formules?
  Aspects syntaxiques
- Comment déterminer la valeur de vérité d'une formule ? Aspects sémantiques
- Existe-t-il un lien entre logique et mathématique? Aspects algébriques (Mr G. Boole)
- Comment démontrer (automatiquement) de nouveaux résultats ? Aspects déductifs

Logique des propositions

Aspects syntaxiques

Les données

Variables propositionnelles

$$P = \{p, q, r, \ldots\}$$

Il s'agit d'**énoncés élémentaires**

Connecteurs logiques

$$C = \{ \neg, \wedge, \vee, \rightarrow, \leftrightarrow \}$$

Logique des propositions

Aspects syntaxiques

F ***l'ensemble des formules du calcul propositionnel***

Definition (recursive)

Toute formule $F \in \mathcal{F}$ est de l'une des formes suivantes

1. $F = p$ *avec* $p \in P$, F *est alors dite formule élémentaire ;*
2. $F = \neg(H)$ *avec* $H \in \mathcal{F}$ *;*
3. $F = (H)\square(K)$ *avec* $\square \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$ *et* $(H, K) \in \mathcal{F}^2$.

Exemples

$(p) \rightarrow ((p) \rightarrow (q))$

Contre-exemples

$p \wedge q$ $\qquad$ $(p)(q)\wedge$

Logique des propositions

Aspects syntaxiques

Règles d'élimination des parenthèses

- Supprimer les parenthèses entourant les variables
- Tenir compte de la priorité des connecteurs ordre standard : $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$
- Considérer qu'un opérateur unaire l'«emporte» toujours sur un opérateur binaire

Exemples

- $(\neg(p)) \wedge (q)$ devient $\neg p \wedge q$
- $((\neg(p)) \wedge (q)) \rightarrow (r)$ devient $\neg p \wedge q \rightarrow r$
- Par contre, $(\neg(p)) \wedge ((q) \rightarrow (r))$ devient $\neg p \wedge (q \rightarrow r)$

Logique des propositions

Aspects sémantiques

Valeurs de vérité

Definition (Logique binaire)

- **vrai** *(noté ■ ou 1)*
- **faux** *(noté □ ou 0)*

Definition (Operateur)

À chaque connecteur c de C, on associe un **opérateur** $\underline{c}$.

L'operateur $\underline{\neg}$

À $\neg$ est associé l'opérateur unaire $\underline{\neg}$ de $\{□, ■\}$ dans $\{□, ■\}$:

- $\underline{\neg}(□) = ■$
- $\underline{\neg}(■) = □$

Logique des propositions

Aspects sémantiques

Table de vérités

Definition (Table de verite)

C'est un tableau dont les lignes sont les interprétations possibles

Tables de verites des operateurs binaires usuels

p	q	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
□	□	□	□	■	■
□	■	□	■	■	□
■	□	□	■	□	□
■	■	■	■	■	■

Logique des propositions

Aspects sémantiques

Definition (Interpretation)

À chaque variable propositionnelle p on associe une **interprétation** *ou valeur de vérité*

$$\delta : P \longrightarrow \{\square, \blacksquare\}$$

Logique des propositions

Aspects sémantiques

Formules particulières

Definition (Formule satisfiable)

Une formule est **satisfiable** *si et seulement si :*

$$\exists \delta \quad \delta(F) = \blacksquare$$

- NB : on dit aussi que F est consistante.

Exemple

$F = (\neg p \rightarrow q) \wedge (q \leftrightarrow r)$ est satisfiable pour $\delta(p) = \square$ et $\delta(q) = \delta(r) = \blacksquare$.

Logique des propositions

Aspects sémantiques

Exemple : *On considère : $(\neg p \rightarrow q) \wedge (q \leftrightarrow r)$*

p	q	r	$\neg p$	$\neg p \rightarrow q$	$q \leftrightarrow r$	$(\neg p \rightarrow q) \wedge (q \leftrightarrow r)$
0	0	0	1	0	1	0
0	0	1	1	0	0	0
0	1	0	1	1	0	0
0	1	1	1	1	1	1
1	0	0	0	1	1	1
1	0	1	0	1	0	0
1	1	0	0	1	0	0
1	1	1	0	1	1	1

Logique des propositions

Aspects sémantiques

Formules particulières

Definition (Antilogie)

Une formule F est une **antilogie** *si et seulement si :*

$$\forall \delta \quad \delta(F) = \square$$

NB : on dit aussi que F est inconsistante, contradictoire, ou encore insatisfiable.

Exemple

$p \wedge \neg p$ est une antilogie.

p	$\neg p$	$p \wedge \neg p$
□	■	□
■	□	□

Logique des propositions

Aspects sémantiques

Formules particulières

Definition (Tautologies)

Une formule F est une **tautologie** *si et seulement si :*

$$\forall \delta \quad \delta(F) = \blacksquare$$

On note $\vdash F$

On note $\mathcal{T}$ *l'ensemble des tautologies. On a :* $\mathcal{T} \subset \mathcal{F}$

Exemple

$p \vee \neg p$ est une tautologie.

p	$\neg p$	$p \vee \neg p$
□	■	■
■	□	■

Logique des propositions

Aspects syntaxiques

Definition (Formules tautologiquement equivalentes)

Deux formules F et G sont **tautologiquement équivalentes** *si et seulement si :*

$$\forall \delta, \delta(F) = \delta(G) \quad \text{ou encore} \quad \forall \delta, \delta(F \leftrightarrow G) = \blacksquare$$

On note $\vdash F \leftrightarrow G$

Exemple

- $p \rightarrow q$ et $\neg p \vee q$ sont tautologiquement équivalentes. On peut donc écrire :

$$\vdash (p \rightarrow q) \leftrightarrow (\neg p \vee q).$$

Logique des propositions

Aspects syntaxiques

Équivalences tautologiques bien connues

double implication

$$\vdash (F \leftrightarrow G) \leftrightarrow ((F \rightarrow G) \wedge (G \rightarrow F))$$

lien implication disjonction

$$\vdash (F \rightarrow G) \leftrightarrow (\neg F \vee G)$$

commutativite

$$\vdash (F \vee G) \leftrightarrow (G \vee F) \qquad \vdash (F \wedge G) \leftrightarrow (G \wedge F)$$

Logique des propositions

Aspects syntaxiques

associativite

$$\vdash (F \vee (G \vee H)) \leftrightarrow ((F \vee G) \vee H)$$

$$\vdash (F \wedge (G \wedge H)) \leftrightarrow ((F \wedge G) \wedge H)$$

distributivite

$$\vdash (F \vee (G \wedge H)) \leftrightarrow ((F \vee G) \wedge (F \vee H))$$

$$\vdash (F \wedge (G \vee H)) \leftrightarrow ((F \wedge G) \vee (F \wedge H))$$

Équivalences tautologiques bien connues

Logique des propositions

Aspects syntaxiques

Équivalences tautologiques bien connues

elements neutres

$$\vdash (F \vee \square) \leftrightarrow F \qquad \vdash (F \wedge \blacksquare) \leftrightarrow F$$

elements absorbants

$$\vdash (F \vee \blacksquare) \leftrightarrow \blacksquare \qquad \vdash (F \wedge \square) \leftrightarrow \square$$

tiers exclu et non-contradiction

$$\vdash (F \vee \neg F) \leftrightarrow \blacksquare \qquad \vdash (F \wedge \neg F) \leftrightarrow \square$$

Logique des propositions

Aspects syntaxiques

Équivalences tautologiques bien connues

double negation

$$\vdash \neg(\neg F) \leftrightarrow F$$

lois de de Morgan

$$\vdash \neg(F \vee G) \leftrightarrow (\neg F \wedge \neg G)$$

$$\vdash \neg(F \wedge G) \leftrightarrow (\neg F \vee \neg G)$$

simplification

$$\vdash (F \rightarrow (G \rightarrow H)) \leftrightarrow ((F \wedge G) \rightarrow H)$$

Logique des propositions

Aspects syntaxiques

Formes normales

Definition (Litteraux)

Les éléments de P sont appelés **littéraux positifs**. *La négation d'un élément de P est un* **littéral négatif**.

Definition (Forme normale disjonctive)

Une formule F est sous **forme normale disjonctive** *si et seulement si*

$$F = \bigvee_{i \in]k]} H_i \quad \text{et} \quad \forall i \in]k], H_i \in (P \cup P_\neg \cup \{\wedge, (,)\})^+$$

- NB : si dans chaque Hi figurent toutes les variables ou leur négation, on parle de forme canonique

Logique des propositions

Aspects syntaxiques

Forme normale disjonctive

Exemple

- $(p \wedge q) \vee (p \wedge r)$ est une formule sous forme normale disjonctive non canonique.
- $(p \wedge q \wedge r) \vee (p \wedge q \wedge \neg r) \vee (p \wedge \neg q \wedge r)$ est une formule sous forme normale disjonctive canonique.

Exercice

Montrer que les deux formules de l'exemple précédent sont tautologiquement équivalentes.

A faire pour le TD

Logique des propositions

Aspects syntaxiques

Forme normale conjonctive

Definition (Forme normale conjonctive)

Une formule F est sous **forme normale conjonctive** *si et seulement si*

$$F = \bigwedge_{i \in]k]} H_i \quad \text{et} \quad \forall i \in]k], H_i \in (P \cup P_\neg \cup \{\vee, (,)\})^+$$

Definition (Clause)

Une clause est une disjonction de littéraux

A retenir : La forme normale conjonctive est aussi appelée forme clausale

Logique des propositions

Aspects déductifs

Notion de conséquence

Definition (Consequence logique)

Soit $\mathcal{A} = \{F_1, \ldots, F_n\}$ un ensemble d'éléments de $\mathcal{F}$ et G une formule.
On dit que G est **conséquence logique** *de $\mathcal{A}$ si et seulement si toute distribution de valeur de vérité satisfaisant simultanément toutes les formules de $\mathcal{A}$ satisfait G.*
On note $\mathcal{A} \vdash G$

Exemple

On a ainsi : $\{p \rightarrow q, p\} \vdash q$ et aussi $\{p \rightarrow q, \neg q\} \vdash \neg p$

Logique des propositions

Aspects déductifs

Notion de conséquence

Theoreme

$\mathcal{A} \vdash G$ *si et seulement si* $\vdash (F_1 \wedge \ldots \wedge F_n) \rightarrow G$.

Theoreme (Raisonnement par l'absurde refutation)

$\mathcal{A} \vdash G$ *si et seulement si* $F_1 \wedge \ldots \wedge F_n \wedge \neg G$ *est inconsistante.*

Logique des Prédicats

- Système formel
- Logique des prédicats de premier ordre
  - présentation et syntaxe
  - Règles d'inférences
  - Sémantique
  - Unification
  - Résolution
  - Stratégie d'utilisation de la résolution
  - Programmation Logique ***Prolog***

Systèmes formels

Un système formel (SF) est constitué :

- Un ensemble de Symboles A (alphabet)
- Un langage L, ensemble de mots (ou formules), avec un procédé de construction des mots du langage (les fbf)
- Des axiomes (mots ou énoncés particuliers)
- Des règles de déductions ou de dérivations ou inférences

Une dérivation (ou preuve ou démonstration) de E est une suite finie d'éléments de L, Soit $E_1, E_2, \ldots, E_n = E$, E_i est un axiome ou bien se déduit des E_j précédents par une règle ; c'est l'ensemble des théorèmes

Systèmes formels

Problème:
Étant donné un énoncé, est ce un théorème?
En général non

Pour certain systèmes formels, des méthodes existes :

ils sont décidables

Exemple de système formel:
Exemple emprunté à hofstader
(Gödel, Escher, Bach : an eternal Golden Brain)
A={M, I, U}
L: ensembles des mots formés sur A
Axiome : MI
Règles :
Règles de productions
mI → mIU
Mm → Mmm
m est un mot quelconque
Règles de réécritures
I I I → U
UU →
MU est-il un théorème?
Remarque : calcul propositionnel

Exemple #1 de système formel

Système (JP) (proposé par Hofstadter) est défini par :

- Alphabet : Z = { a, b, □ }
- Mots = suite de symboles quelconques de Z
- Axiome : a□a
- Règle : R1) c1□c2 → bc1□bc2

a,b : constantes

□ : opérateur

c1, c2: variables substituables par n'importe quelle suite de symboles a ou b

bbba□bbba est un théorème.

baa□, □a□b, aabb□a sont des formules mais non des théorèmes.

Exemple #1 de système formel

Première interprétation :

- a ↔ 0

- b ↔ successeur d'un nombre sur l'ensemble des entiers

- □ ↔ signe d'égalité dans le même ensemble

A s'interprète 0 = 0. Les théorèmes successifs s'écrivent :

1 = 1

2 = 2

- L'interprétation est correcte dans l'arithmétique usuelle.

Exemple #1 de système formel

Deuxième interprétation :

a ↔ Socrate est mortel

b ↔ la négation

□ ↔ l'identité de proposition

A s'interprète « Socrate est mortel » est identique à « Socrate est mortel » : énoncé vrai.

ba□ba s'interprète « Socrate n'est pas mortel » est identique à

« Socrate n'est pas mortel » : énoncé vrai.

bba□a n'est pas un théorème, bien que « La négation de Socrate

n'est pas un mortel est identique à Socrate est mortel » est un énoncé vrai dans notre interprétation.

Exemple #2 de système formel

Le système (PG) de D. Hofstadter est défini par

Alphabet : Z = { p, g, - }

- Mots : toute suite de lettres de Z
- Axiome : xp-gx-
- avec x composé d'un ou plusieurs tirets
- Règle d'inférence : R1) xpygz → xpy-gz-

x, y, z sont des variables.

Exemples de théorèmes :

--p---g-----

Limitations des systèmes formels

Les premières limitations des systèmes formels ont été posées

par les théorèmes de Gödel, Tarski et Church :

Gödel (1930) :

« Il existe des systèmes formels contenant des mots m tels que ni m ni ¬m ne sont démontrables. »

Tarski (1935) :

« Il existe des systèmes formels pour lesquels toute interprétation conduit à des énoncés à la fois vrais et non démontrables. »

Church (1936) :

« Il existe des systèmes formels indécidables, c'est-à-dire qui ne disposent pas d'une procédure pour séparer les théorèmes des non théorèmes. »

Logique des prédicats (1èr ordre)

- Langage formel
- Exprime des Connaissances complexes avec rigueur
- Exprime les méthodes de combinaison des connaissances qui engendrent d'autres

Logique des prédicats
Présentation est Syntaxe

Alphabet : ensemble de symboles

- Séparateur : , ( )
- Constantes : les caractères latins minuscules, chiffres
- Variables : les caractères latins majuscules
- Prédicats : est une relation muni d'une *arité* « n »

  Nom-prédicats (, ,)

  exple Produit (2, 4, 8)

  ➢Si n = 0 alors proposition
- fonction : une application d'un :

  $D^n \rightarrow D$

  munie d'un nom minuscule et d'une arité « n »

  exple f(X), g(X,Y).
- Connecteurs : $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$
- Quantificateurs : $\forall$, $\exists$

Logique des prédicats
Définitions

Terme:

- une constante
- Une variable
- $f(t_1,\ldots,t_n)$

  f : symbole fonctionnel

  t_i : des termes

  plus (2,3)

Atome

- Proposition (n=0)
- $P(t_1,\ldots,t_n)$

  P : un symbole de prédicat

  ti : des termes

  Entre (table, mur, chaise)

Formule bien formée (fondée) fbf :

- Atome est une fbf
- Si G et H sont des fbf alors

  $\neg$ G , H $\vee$ G, H $\wedge$ G, G $\rightarrow$ H,

  G $\leftrightarrow$ H sont des fbf
- SI G est une fbf, X est une variable :

  ($\exists$ X) G et ($\forall$ X) G sont des fbf

Remarque

Logique des prédicats de 1èr ordre est associé à la définition des fbf qui interdit de quantifiers les prédicats

Exemple:

$$(\exists X)\,(\forall Y)\,(P(X,Y) \vee Q(X,Y)) \rightarrow R(X))$$
$$(\neg(P(a) \rightarrow P(b)) \rightarrow \neg\, P(b)$$

Contres exemples

$$(\exists P)\, P(a)$$
$$(\forall f\,)\,(\exists X)\, P(f(X),b)$$

Règles d'inférences

Est une représentation d'un procédé qui, à partir d'un ou plusieurs fbf dérivent d'autres fbf:

Modus ponens :

$$\frac{G \rightarrow H \quad G}{H}$$

Spécialisation universelle

$$\frac{\forall X\ G(X) \quad a \text{ une constante}}{G(a)}$$

Exemple

« Tous les hommes sont mortels »

$\forall X$ HOMME $(X) \rightarrow$ MORTEL (X)

$$\frac{\text{HOMME (socrate)}}{\text{HOMME (socrate)} \rightarrow \text{MORTEL (socrate)}}\ \text{Spe}$$

$$\frac{\text{HOMME (socrate)} \quad \text{HOMME (socrate)} \rightarrow \text{MORTEL (socrate)}}{\text{MORTEL(socrate)}}\ \text{M.P}$$

Modus Tollens:

$$\frac{G \rightarrow H \qquad \neg H}{\neg G}$$

$$(G \rightarrow H) \leftrightarrow (\neg H \rightarrow \neg G)$$

$$\frac{\forall X \; HOMME(X) \rightarrow MORTEL(X) \qquad \neg MORTEL(dieu)}{\neg HOMME(dieu)}$$

Axiomes : fbfs initiales

Théorèmes : fbfs obtenues

Preuve : ens d'inférences conduisant aux théorèmes

Sémantique

Examiner les bases qui permettent d'utiliser Des fbf pour présenter et raisonner sur les « valeurs de vérité ».

Interprétation d'une fbf:

- Choix d'un domiane D
  - ✓ Ensemble non vide d'éléments
- Assigner à chaque constante un élément de D
- Assigner à chaque proposition un élément de l'ensemble {Vrai, Faux}
- Un prédicat d'arité «n ⌑1» est une application: $D^n \rightarrow$ {Vrai, faux}
- Une fonction d'arité «n ⌑1» est une application $D^n \rightarrow D$

EXEMPLE

- G1: P(X)
- G2: $(\forall X)(\exists Y)\, Q(X,Y)$
- G3: $(\forall X)(R(X) \rightarrow T(f(X),a))$

I_1: $D_1=\{1,2\}$

P(1)	P(2)
F	V

I_2: $D_2=\{1,3\}$

Q(1,1)	Q(1,3)	Q(3,1)	Q(3,3)
F	V	F	F

I_3: $D_3=\{4,5\}$

a
4

$f(4)$	$f(5)$
5	4

R(4)	R(5)
V	F

T(4,4)	T(5,4)
V	V

- Si G est de la forme $\forall$ X G'(X) alors :
  G est vrai si pour toute valeur de X G'(X) est vrai

- Si G est de la forme $\exists$ X G'(X) alors :
  G est vrai s'il existe au moins une valeur de Y telle que G'(Y) est vrai

Table de vérité

Valeur de vérité selon une interpétation

G	G'	$\neg$ G	G $\wedge$ G'	G $\vee$ G'	G $\rightarrow$ G	G $\leftrightarrow$ G'
F	F	V	F	F	V	V
F	V	V	F	V	V	F
V	F	F	F	V	F	F
V	V	F	V	V	V	V

Validité:

Une formule est valide ssi sa valeur est vraie $\forall$ I
Sinon elle est invalide
D'où la notion de « tautologie »:

$$\forall X\ P(X) \vee \neg P(X)$$

Inconsistance

Une formule est inconsistante ssi sa valeur est fausse pour toute interprétation:
✓Formule contradictoire
Si non elle est consistante

$$\forall X\ P(X) \wedge \neg P(X)$$

Satisfiabilité

Une formule est satisfiable dans une interprétation I si elle est vraie dans cette interprétation
Si non elle est insatisfiable
Si une formule est vraie selon une interprétation on dit que I est un modèle de la formule

Exemple

- G_1: $\forall X\ P(X) \vee \neg Q(X)$
- G_2 : $\forall X\ P(X) \vee \neg P(X)$
- G_3: $\forall X\ P(X) \vee ((\exists Y) \neg P(Y))$

- G_1 est consistante et satisfiable dans I_1

  I_1: D={1} P(1)|V Q(1)|V

- G_1 est insatisfiable dans I'_1

  I'_1 : D={1} P(1)|F Q(1)|V

- G_2 est valide
- G_3 est inconsistante

Exercice

a) Soit l'énoncé G : « Toute ville du Maroc a un maire qui a été mordu par tous les chiens de la ville »:

Représenter G par une formule de LPP.
On utilisera les prédicats
Ville (X) , Maire (X,Y), Chien(X,Y), Amordu(X,Y)
G est il vrai?

b) Soit l'énoncé G': « Tout nombre entier naturel x à un successeur qui est inférieur ou égal à tout entier supérieur à x ».

Représenter G' par une formule de LPP.
On utilisera les prédicats:
Entiers (X), Successeur(X,Y), Entiersuperieur (X,Y),
Y ¤ X (X est inférieur à Y)
G' est il vrai?

c) Déduire des questions a) et b) que la formule O:
$\forall X (P(X) \rightarrow \exists Y(Q(X,Y) \wedge \forall Z (R(Z,X) \rightarrow S(Z,Y)))$
est satisfiable non valide

Formule équivalente

2 fbf G et H sont équivalentes ssi elles prennent les mêmes valeurs (V ou F) pour toute interprétation:

$(\forall I)\ (I(G)=I(H))$

Exemple

G: $P(a) \rightarrow Q(b)$

H: $(\neg P(a)) \vee Q(b)$

P(a)	$\neg$ P(a)	Q(b)	P(a) $\rightarrow$ Q(b)	$\neg$ P(A) $\vee$ Q(b)
V	F	V	V	V
V	F	F	F	F
F	V	V	V	V
F	V	F	V	V

Conséquence logique

G une fbf est une conséquence logique des S= {H1,H2,….Hn} si toute interprétation satisfait S Satisfait aussi G

Remarque

Conséquence logique 🕒Théorème

Le théorème est le résultat d'une preuve (ensemble D'application de règles d'inférences)

Supposons que nous avons un système de règles D'inférences :
Nous disons que ces règles sont saines :
si tout théorème Dérivable de tout ensemble de fbf est lui aussi une conséquence logique de cet ensemble de fbf.

Unification

Substitution:

Est un ensemble de couple(X|ti) où X est une Variable et ti un terme quelconque du calcul Des prédicats

A: $P(U) \rightarrow R(X)$

$\sigma = \{(X|f(a))(U|f(X)\}$

$A\,\sigma : \; P(f(f(a))) \rightarrow R(f(a))$

$\sigma' = \{(U|f(X))(X|f(a)\}$

$A\,\sigma' : \; P((f(X)) \rightarrow R(f(a))$

$A\,\sigma \neq A\,\sigma'$

Unificateur :

σ est un unificateur de 2 formules A_1 et A_2 si

$$A_1 \sigma = A_2 \sigma$$

A_1 : P(X,Z) A_2 : P(f(Y),g(a))

$$\sigma = \{X|f(U))(Y|U)(Z|g(a))\}$$

$A_1 \sigma$ = P(f(U),g(a))
$A_2 \sigma$ = P(f(U),g(a))

Unificateur plus général σ_2 s'il existe une substitution γ telle que $\forall$ σ_1 unificateur:

$$\sigma_1 = \gamma \sigma_2$$

$$\sigma_1 = \{X|f(U))(Y|U)(Z|g(a))\}$$

$$\sigma_2 = \{X|f(V))(Y|V)(Z|g(a))(U|V)\}$$

$$\sigma_1 = (V \mid U) \sigma_2$$
$$\sigma_2 = (U \mid V) \sigma_1$$

Algorithme d'unification de 2 atomes A et B

- $\theta := \varnothing$
- Tant que $A\theta \neq B\theta$

faire

• déterminer le symbole le + à gauche de $A\theta$ différent du symbole de même rang de $B\theta$.

•Déterminer t1,t2 sous termes de $A\theta$ et $B\theta$ Qui commencent à ce symbole.

•Si « aucune n'est une variable »

Ou « l'un est une variable incluse dans l'autre »

Alors A et B ne sont pas unifiable

Sinon faire

• Déterminer X une variable parmi t1 et t2

• Déterminer t de t1 et t2 ≠X

• $\theta := (X|t)\theta$

finSi

Fin tant que

θ Unificateur le + général de A et B

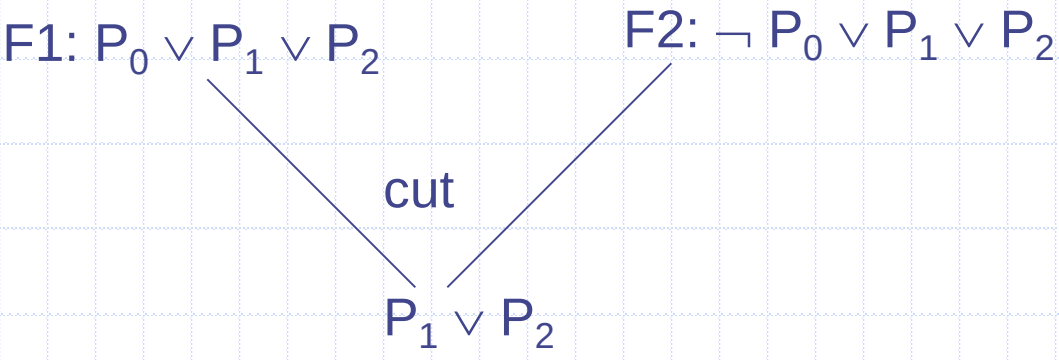
Résolution sans variable (RSV)

$A_0, A_1, \ldots, A_n$ des formules du calcul propositionnel
On cherche à savoir si $A_0, A_1, \ldots, A_n$ est satisfiable c. à d.
S'il existe I tq $[I(A_0), \ldots I(A_n)]$ =vrai

- Table de vérité 2^n
- Résolution est simple et généralisable aux formules des prédicats du 1er order

EXEMPLE

F1: $P_0 \vee P_1 \vee P_2$
F2: $\neg P_0 \vee P_1 \vee P_2$
F3: $\neg P_1 \vee P_2$



Résolution avec variable RAV

Extension de la RSV sur le calcul des prédicats

La méthode de résolution fournit plusieurs algorithmes Pour déterminer si une formule résulte d'un ensemble Fini de formules.

Efficacité des méthodes de résolution permet une utilisation réelles de ces algorithmes pour la D.A.T.

Nombreuses Stratégies existent

Système formel de la RAV

A_{RAV}= Alphabet du calcul des prédicats du premier ordre

fbf_{RAV} = {ens des clauses
$P_{i1} \vee P_{i2} \vee \ldots P_{in} \vee \neg P_{j1} .. \neg P_{jm}$
p_k:formule atomique} $\cup$ $\square$ (clause vide)

$A_{RAV} = \varnothing$

R_{RAV} ={résolution, diminution}

Résolution

H,G $|---_{res}$ K
ssi
$H = H_1 \vee H_2$
$G = \neg G_1 \vee G_2$
$K = (H_2\,\theta \vee G2)\,\sigma$ résolvant de H et G

θ : une substitution de renomage
σ : l'unificateur le plus général de $H_1\,\theta$ et G_1

$H_1\,\theta\,\sigma = G_1\,\sigma$

Exemple 1

$H = P(X,c) \vee R(X)$

$G = \neg P(c,c) \vee Q(X)$

$\theta = \{(X|Y)\}$

$H\theta = P(Y,c) \vee R(Y)$

$\sigma = \{(Y|c)\}$

$H\theta\sigma = P(c,c) \vee R(c)$

$G\sigma = \neg P(c,c) \vee Q(X)$

$R(c) \vee Q(X)$ res

Exemple 2

$H = P(X,f(X))$

$G = \neg P(a,Y) \vee R(f(Y))$

$R(f(f(a)))$ res

$\theta = \varnothing$

$\sigma = \{(X|a)(Y|f(X))\}$

Diminution

$H \vdash_{dim} G$
Ssi
$H = H_1 \vee H_2 \vee H_3$
$G = H_1 \sigma \vee H_3 \sigma$
σ unificateur le + général de H_1 et H_2
$H_1 \sigma = H_2 \sigma$

Exemple 1

$H = P(X,g(Y)) \vee P(i(c), Z) \vee R(X,Y,Z)$

$\vdash$——————————dim

$P(i(c), g(Y)) \vee R(i(c),Y,g(Y))$

$\sigma = \{(X|i(c)), (Z|g(Y))\}$

Propriétés du principe de résolution

Le principe de résolution est une règle saine
c à d tous les théorèmes obtenus sont conséquences
Logiques des axiomes.

$$H \vdash C \Rightarrow H \vdash C$$

Si l'on obtient la clause vide □, c à d $H \vdash \square$, c'est que
L'ensemble de départ est inconsistant.

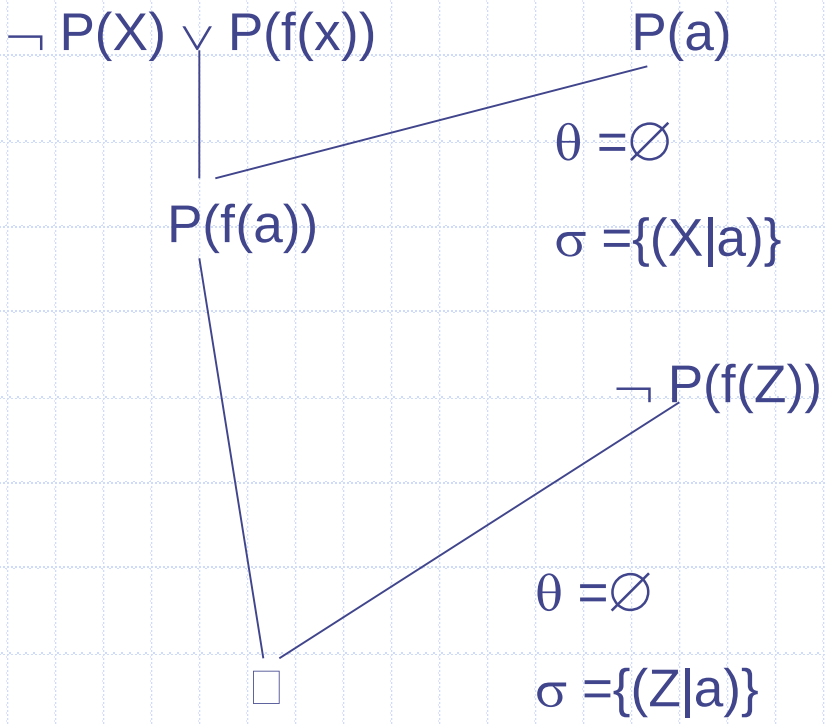
Donc pour monter que $H \vdash C$, on cherchera à déduire
La clause vide de $G=H \cup \{\neg C\}$.
On obtiendra alors une réfutation.

Si un ensemble de formules est inconsistant, on peut
Déduire la clause vide. Le principe de Résolution
Est alors dit complet pour la réfutation

Exemple de déduction dans un système RAV

On cherche à déduire □ :

f1: ¬ P(X) ∨ P(f(x))
f2:P(a)
f3: ¬ P(f(Z))



f1,f2 et f3 sont contradictoire

Exemple

1. Pour tout crime, il y a quelqu'un qui l'a commis
2. Seuls les gens malhonnêtes commettent des crimes.
3. Ne sont arrêtés que les gens malhonnêtes
4. Les gens malhonnêtes arrêtés ne commettent pas de crimes
5. Il y a des crimes
6. Établir que B: « il y a des gens malhonnêtes non arrêtés »

Mise en Forme Clausale

Mise en forme Prenex

a. Eliminer les connecteurs $\rightarrow$ et $\leftrightarrow$
b. Accorder les connecteurs $\neg$ aux atomes
c. Distinguer les variables de sorte que chaque quantificateur gouverne une variable
d. Déplacer tous les quantificateurs à gauche de la formule (sans changer l'ordre)

Mise sous forme clausale

- Eliminer $\exists$
  1. $\exists$ X P(X) $\Rightarrow$P(a)
  2. $\forall$ X $\exists$ Y (P(X,Y)) $\Rightarrow$ $\forall$ X (P(X,f(X))
     - f: fonction skolem
     - f(X) : constante skolem
- Eliminer tous les quantificateurs
- Passer sous forme normale conjonctive(clause)

Exemple

1. $\forall X\ Cr(X) \rightarrow \exists Y\ Co(Y,X)$
2. $\forall Y \forall X\ ((Cr(X) \wedge Co(Y,X)) \rightarrow Ma(Y)$
3. $\forall Y\ Ar(Y) \rightarrow Ma(Y)$
4. $\forall Y\ ((Ma(Y) \wedge\ Ar(Y)) \rightarrow \neg(\exists X\ (Cr(X) \wedge Co(Y,X)))$
5. $\exists X\ Cr(X)$
6. $\neg(\exists Y(Ma(Y) \wedge \neg\ Ar(Y)))$

1. $\neg\ Cr(X) \vee Co(f(X),X)$
2. $\neg\ Cr(X) \vee \neg\ Co(Y,X) \vee Ma(Y)$
3. $\neg\ Ar(Y)\ \vee Ma(Y)$
4. $\neg\ Ma(Y) \vee \neg\ Ar(Y) \vee \neg\ Cr(X) \vee \neg\ Co(Y,X)$
5. $Cr(a)$
6. $\neg\ Ma(Y) \vee Ar(Y)))$

B est une conséquence de 1,2,3,4 et 5
Résolution par réfutation

Méthode générale de réfutation par résolution

- Cette méthode, dans sa généralité, n'est pas déterministe.

Pour monter H |--- C,

Mettre H∪ {¬ C} sous forme de clauses.
Soit G l'ensemble de clauses;
Tant que □ ∉G
    choisir deux clauses de G;
    si elles ont des résolvantes, en choisir un et l'ajouter à G
Si □ ∈G, succès

La complétude du principe de résolution affirme que si H |--- C, il existe une réfutation, elle ne dit pas que l'on pourra l'obtenir à coup sûr n'importe comment

Stratégies de résolution

Définir des critères pour choisir les deux clauses et le résolvant :

- restreindre les choix
- ordonner les choix



Divers stratégies

S est complète si elle garantit que, s'il existe une Réfutation, alors il en existe une satisfaisant les Critères de choix de S:

Soit S une stratégie

$R_S(G)$: la réunion d'un ensemble G de clause et de tous les résolvantes de G obtenus en appliquant S

$$R_S^{P+1}(G) = R_S(R_S^P(G))$$

S est complet ssi pour tout ensemble G inconsistant, $\exists\, n \in N$ tel que $\square \in R_s^n(G)$

Graphe de dérivation, de réfutation, de recherche

Le graphe de dérivation a pour sommets toutes les clauses qui peuvent être déduites reliées aux Clauses dont elles sont issues

Exemple

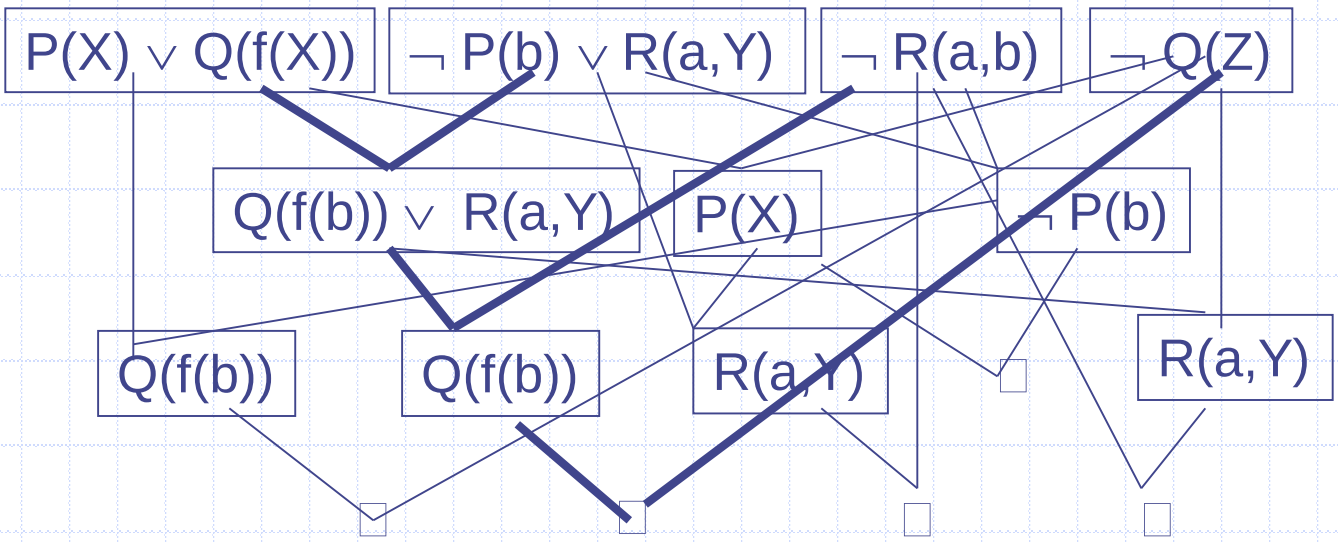
H= {$\forall$ X $\exists$ Y ($\neg$ P(X) $\rightarrow$ Q(Y)), $\forall$ Y(P(b) $\rightarrow$ R(a,Y)}

On veut prouver que $\neg$ R(a,b) $\rightarrow$ $\exists$ Z Q(Z)

On construit l'ensemble de clauses

0. G= {P(X) $\vee$ Q(f(X)), $\neg$ P(b) $\vee$ R(a,Y), $\neg$ R(a,b) , $\neg$ Q(Z) }

- R(G)=G$\cup$ {Q(f(b)) $\vee$ R(a,Y), P(X) , $\neg$ P(b) }
- R^2(G)=R(G) $\cup${Q(f(b)) , R(a,Y), $\square$ }
- R^3(G)=R^2(G)



Stratégie en largeur

- On définit la profondeur d'une clause
- Les clauses initiales ont une profondeur nulle
- Les résolvantes de deux clauses dont l'une est à une profondeur n et l'autre à une profondeur ≤ n, sont à la profondeur n+1

À ne chercher les clauses de profondeur n+1 que Lorsqu'on a obtenu toutes les clauses de profondeur n.

☞ Directement complète

Stratégie positive

On impose de choisir une clause positive à chaque résolution

☞ complète

Stratégie positive + largeur

☞ Directement complète

Programmation logique Prolog

Introduction

Prolog est un langage lié:

- à la logique formelle;
- à une modélisation du raisonnement;
- au traitement linguistique, à l'écriture du grammaires, d'analyseurs;
- transcrire du SQL en prolog et d'interfacer une base de données avec un module logique écrit en prolog.

Les éléments fondamentaux du prolog

1 Les faits

Sont des affirmations qui décrivent des relations ou des propriétés:

élève (ali, 1975, nfo, 2).
masculin(ali).
père(ali,Mohamed).
féminin (aicha).
Mère(ali ,aicha).

Forme général d'un fait: prédicat (argument1,argument2,….)

2 Les questions ou les requêtes

Une fois le programme chargé, on peut poser des questions sur les faits

? Masculin (ali).
Yes
? Masculin (jawad)
NO

Les éléments fondamentaux du prolog

3 les types de prolog

- Les constantes

  Des chaînes alphanumériques qui commencent par une minuscule

  Les nombres 19,-25,

- Les variables

  Commencent par une majuscule ou le signe _.

  _ tout seul est une variable anonyme:

4 les structures ou termes composés:

Se compose d'un foncteur avec une suite d'arguments. Les arguments peuvent être des atomes, des nombres, des variables, ou bien des structures:

élève(ali, 1975, info, 2, adresse(6, 'Route de rabat', 'Casa'))

Les éléments fondamentaux du prolog

5 Les règles

Les règles permettent d'exprimer des conjonctions de buts. Leur forme général est: Tête :- C1,C2,…,Cn
La tête de la règle est vrai si chacun des éléments de la règle C1,C2,…,Cn est vrai. (Clauses de Horn)

fils (A,B):-père(B,A),masculin(A).

fils (A,B):-mère(B,A), féminin(A).

parent(X,Y):-père(X,Y).

parent(X,Y):-mère(X,Y).

grand_parent(X,Y):- parent(X,Z),parent(Z,Y).
ancêtres(X,Y)??

Les éléments fondamentaux du prolog

6 Les listes

Une liste est une structure de la forme:

[a]
[a, b]
[a, X, adresse(X, casa)]
[[a,b],[c,d]]

La notation « | » permet d'extraire la tête et la queue de la liste:

? [a,b]=[X|Y]
X=a, Y=[b]

? [a]=[X|Y]
X=a, Y=[]

Les éléments fondamentaux du prolog

Quelques prédicats de manipulation de listes

Appartenance

```
app(X,[]):-fail.
app(X,[X|_]).
app(X,[_|L]):-app(X,L).
```

Longueur

```
long([],0).
long ([X|L]),N):- long(L,M), N is M+1.
```

Intersection

```
intersection([] ,_, [])
intersection([X|L1] ,L2, [X|L3]):-
app(X,L2),intersection (L1,L2,L3).
intersection([_|L1] ,L2, L3):-
intersection (L1,L2,L3).
```

Le raisonnement en prolog

R1: est_de bonne_humeur (X):- a_de_l_argent(X),

est_en_vacances(X), il_y_du_soleil

R2: est_de bonne_humeur (X):-
reussit_dans_le_travaille(X),

reussi_dans_sa_famille(X).

R3: a_de_l_argent(ali).

R4: a_de_l_argent(aicha).

R5: est_en_vacances(ali) :- on_est_en(aout).

R6: est_en_vacances(aicha):- on_est_en(juillet).

R7: on_est_en(juillet).

R8: il_y_du_soleil:- on_est_en(aout).

R9: reussit_dans_le_travaille(ali)

R10: reussit_dans_le_travaille(aicha)

R11: reussi_dans_sa_famille(aicha).

Le raisonnement en prolog

Le but : est_de_bonne_humeur (X):

- Prolog va explorer toutes les possibilités d'unification en parcourant un arbre ET/OU en profondeur d'abord. Lorsqu'il aboutit à un échec, il remonte au dernier choix.

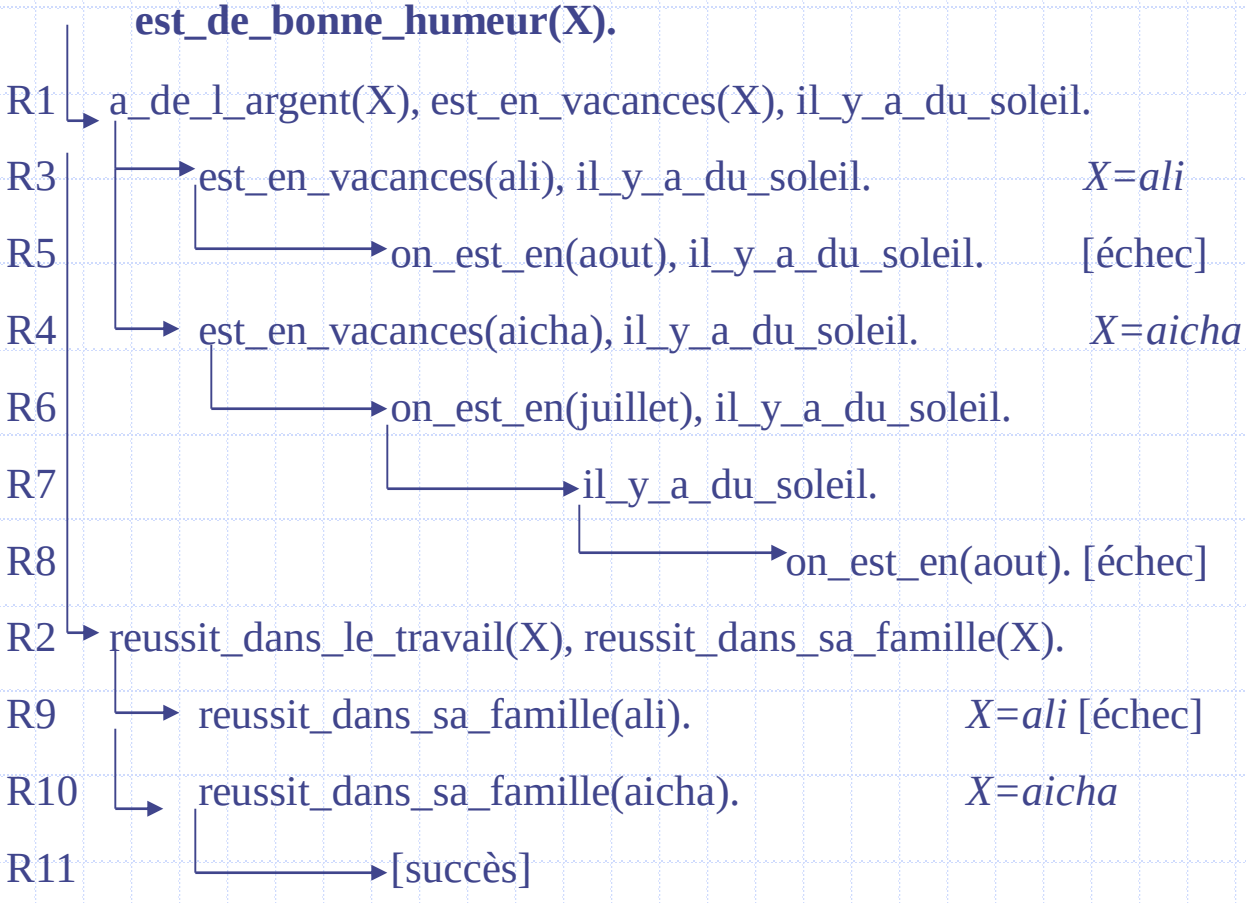
- Un point de raisonnement correspond à une liste de buts et à une liste de substitutions.
- Une règle comprend un membre gauche MG et un membre droit MD (qui peut être vide).
- En un point de raisonnement, ne s'occuper que du but le plus à gauche (le but courant).
- Pour satisfaire le but courant BC :
  - Rechercher la première règle R non encore essayée dont le MG est unifiable avec BC.
  - Remplacer, dans la liste de buts, BC par le MD de R en effectuant les substitutions.
    - Si la règle n'a pas de MD, on enlève BC de la liste de buts : *effacement*.
  - Propager les substitutions issues de l'unification de BC dans la liste de buts.

Le raisonnement en prolog

- En cas de blocage (plus aucune règle n'est applicable), revenir en arrière au plus récent point de raisonnement où une règle pouvait être encore appliquée : *backtracking*.

- La résolution est terminée lorsque la liste de buts est vide ou lorsque plus aucune règle ne peut être appliquée lors du *backtracking* (on revient alors au point de départ).

Arbre de recherche



Le raisonnement en prolog

Introduction de la négation

Prolog possède un *méta-prédicat* not/1 dont l'unique argument doit être un but. Comme on raisonne en monde fermé (ce qui ne peut pas être prouvé est considéré comme étant faux),

un but **not(P)** est réussi lorsque la résolution du but P échoue (négation par l'échec) :

– le but **not(P)** est **satisfait** si le raisonnement appliqué au but P conduit à un **échec**.

– le but **not(P)** est **insatisfait** si le raisonnement appliqué au but P donne un **succès**.

Un not ne peut intervenir que dans le membre droit d'une règle de réécriture :

interdit : not(fort(X)) :- petit(X).

permis : fort(X) :- sportif(X), not(petit(X)).

Coupe ou Cut

Introduction du coupe-choix

• On appelle *coupe-choix* ou *cut* un prédicat prédéfini noté "!" dont l'usage est délicat en Prolog.

•le *cut* agit sur la façon dont l'arbre de raisonnement est parcouru.

```
A( ... ) :-B( ... ), C( ... ).
```

Pour prouver A Prolog peut utiliser toutes les règles prouvant B, toutes règles prouvant C, voire d'autres rè-gles prouvant A.

```
A( ... ) :-!, B( ... ), C( ... ).
```

si toutes les possibilités pour prouver B et C ont échouées, Prolog ne cherchera pas d'autres règles prouvant A

```
A( ... ) :-B( ... ), !, C( ... ).
```

Prolog pourra revenir sur la preuve de C mais pas sur celle de B.

Coupe ou Cut

Exemple

```
q(a).
q(b).
q(c).
r(b,b1).
r(c,c1).
r(a,a1).
r(a,a2).
r(a,a3).
p(X,Y) :-q(X), r(X,Y).
p(d,d1).

p(X,Y) :-q(X), r(X,Y). 6 solutions
p(X,Y) :-q(X), r(X,Y),!. 1 solutions
p(X,Y) :-q(X),!, r(X,Y). 3 solutions
p(X,Y) :-!,q(X), r(X,Y). 5 solutions
```

Récursivité

factoriel (N,F)
Répondre vrai si N*(N-1)*…*3*2*1=F

factoriel(1,1).
factoriel(N,Res)if N>0,N1=N-1,factoriel(N1,FacN1),
Res=N*FacN1.

factoriel(2,Rep) if 2>1, N1=2-1, factoriel(1,FacN1),
Rep=2*FacN1.
On évalue le but factoriel (1,FacN1)
Or nous avons le fait factoriel (1,1)
Donc le but est satisfait
Le but suivant Rep=2*FacN1est satisfait
si résultat est lié à la valeur 2